МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Российский экономический университет имени Г.В. Плеханова»

Пермский институт (филиал)

Сопроводительная записка

по производственной практике

к программному модулю «Модуль импорта товаров в электронный магазин из CSV-файлов»

ПП.02.01 Технология разработки программного обеспечения
(индекс по РУП и наименование практики)

Профессионального модуля ПМ.02 Осуществление интеграции программных модулей
(индекс по РУП и наименование профессионального модуля)

Специальность 09.02.07 Информационные системы и программирование

Квалификация программист

Обучающийся _____ Ощепков Михаил Алексеевич
(подпись) (фамилия, имя, отчество)

Группа __ИПо-42_____

Руководитель практической подготовки от института

_____ Берестов Дмитрий Борисович
(подпись) (фамилия, имя, отчество)

«13» ноября 2026 года

Пермь, 2026 год

ОГЛАВЛЕНИЕ

1. ЦЕЛЬ И ЗАДАЧИ ПРАКТИКИ 3
2. ОПИСАНИЕ ЗАДАНИЯ 4
2.1. Назначение модуля 4
2.2. Требования к CSV-файлу 4
2.3. Требования к функциональности 4
2.4. Ограничения реализации 5
2.5. Существующий процесс и процесс после внедрения 5
2.6. Техническое задание и требования 6
3. АРХИТЕКТУРА РЕШЕНИЯ 7
3.1. Структура проекта 7
3.2. Роли компонентов 7
3.3. План разработки 9
4. ФОРМАТ CSV-ФАЙЛА 10
5. РЕАЛИЗАЦИЯ ФУНКЦИОНАЛЬНОСТИ 11
5.1. Чтение CSV-файла 11
5.2. Проверка данных 12
5.3. Добавление товаров в память 13
5.4. Передача результата в представление 13
5.5. Импорт при запуске приложения 16
6. ТЕСТИРОВАНИЕ И ОТЛАДКА 18
6.1. Тестовые сценарии 18
6.2. Модульные тесты 19
6.3. Отладка 19
6.4. Самопроверка кода 20
7. ПОРЯДОК ЗАПУСКА 22
8. ВЫВОДЫ 23

1. ЦЕЛЬ И ЗАДАЧИ ПРАКТИКИ

Цель производственной практики ПП.02.01 — получение практического опыта разработки требований к программному модулю, проектирования и реализации ASP.NET MVC-приложения, работы с CSV-файлами и коллекциями в памяти, отладки, тестирования и проверки качества кода. Индивидуальное задание выдано руководителем практической подготовки от института Берестовым Д.Б. и выполнялось в период практики с 3 по 13 ноября 2026 года.

В рамках индивидуального задания (вариант 1 «Электроника») решались следующие задачи:

1. изучить требования к модулю импорта товаров и подготовить описание формата CSV-файла;

2. спроектировать модель товара Product, результат импорта ImportResult и хранилище товаров в памяти;

3. реализовать чтение CSV-файла, валидацию данных и загрузку корректных товаров в коллекцию;

4. сделать страницы запуска импорта и просмотра списка товаров;

5. обработать ошибки: отсутствующий и пустой файл, неверные заголовки, неверная цена, отрицательный остаток, дубликаты, пустое название;

6. подготовить тестовые сценарии, модульные тесты и выполнить отладку;

7. разместить код в Git-репозитории с осмысленными коммитами.

2. ОПИСАНИЕ ЗАДАНИЯ

2.1. Назначение модуля

Модуль предназначен для наполнения каталога электронного магазина: он читает CSV-файл с товарами, проверяет каждую строку, добавляет корректные товары в коллекцию в памяти и показывает результат импорта со списком ошибок по номерам строк. Для тестовых данных выбрано сетевое и абонентское оборудование: Wi-Fi роутеры, ТВ-приставки, камеры видеонаблюдения, устройства умного дома. Требования к модулю подробно оформлены в отдельном документе «Техническое задание» по ГОСТ 19.201-78.

2.2. Требования к CSV-файлу

- текстовый файл с расширением .csv в кодировке UTF-8 (дополнительно поддерживается Windows-1251);

- первая строка — заголовки колонок, каждая следующая строка — один товар;

- разделитель — запятая (дополнительно поддерживаются точка с запятой и табуляция);

- значение, содержащее разделитель, заключается в двойные кавычки;

- состав полей и правила их проверки приведены в разделе 4.

2.3. Требования к функциональности

Обязательные функции (п. 3.1 и 11.1 задания):

- импорт CSV-файла по указанному пути, а также автоматически при запуске приложения (путь задается в appsettings.json);

- построчное чтение файла и преобразование строк в объекты Product;

- проверка данных и добавление корректных товаров в коллекцию в памяти;

- вывод результата: всего строк, успешно импортировано, ошибки, список ошибок «Строка — Описание»;

- страница списка загруженных товаров; повторный импорт без падения приложения.

Дополнительно реализованы все 10 элементов из п. 11.2 задания: загрузка файла через браузер; поиск по названию; фильтр по категории; сортировка по цене, остатку и названию; страница деталей товара; подсчет общего количества и стоимости; экспорт в CSV; модульные тесты xUnit; запись ошибок в файл журнала; поддержка разделителей «,», «;» и табуляции.

2.4. Ограничения реализации

- база данных не используется: товары хранятся в List<Product> и теряются после перезапуска (допускается заданием);

- при повторном импорте товар с существующим артикулом пропускается с ошибкой, обновление товаров не выполняется;

- поле CSV не может содержать перевод строки внутри кавычек;

- размер файла, загружаемого через браузер, — не более 5 МБ, принимаются только файлы .csv;

- приложение однопользовательское, разграничение доступа не реализуется.

2.5. Существующий процесс и процесс после внедрения

Описание процесса «AS IS» (как есть). Без модуля импорта товары добавляются в каталог вручную: получить прайс-лист от поставщика; открыть форму добавления товара; перепечатать артикул, название, цену, остаток, бренд и гарантию; визуально проверить значения и сохранить; повторить для каждого товара; при обнаружении ошибки найти товар и исправить его вручную. Ввод большого количества товаров занимает много времени, возникают опечатки в ценах и остатках, один и тот же товар может быть добавлен повторно.

Описание процесса «TO BE» (как будет). Прайс-лист сохраняется в формате CSV; на странице «Импорт» указывается путь к файлу или файл выбирается в браузере; модуль сам читает и проверяет все строки, добавляет корректные товары и пропускает дубликаты; пользователь видит итог и список ошибок с номерами строк, исправляет их в файле и повторяет импорт.

Сравнение процессов приведено в таблице 1.

Таблица 1 – Ручное наполнение каталога и импорт из CSV

Действие	AS IS (вручную)	TO BE (модуль импорта)
Перенос данных	Каждый товар вводится в форму вручную	Все товары загружаются из файла одной операцией
Проверка цены и остатка	Визуально, ошибки пропускаются	Автоматически по правилам валидации
Контроль дубликатов	Отсутствует или вручную	Повторный артикул пропускается с сообщением
Поиск ошибки	Неизвестно, где ошибка	Указан номер строки и описание проблемы
Повторная загрузка	Риск задвоения товаров	Безопасна: дубликаты не добавляются

2.6. Техническое задание и требования

Техническое задание на модуль оформлено отдельным документом по ГОСТ 19.201-78 (папка docs репозитория). Функциональные требования к модулю приведены в таблице 2; каждое требование проверяется тестовым сценарием или модульным тестом (раздел 6).

Таблица 2 – Функциональные требования

№	Требование	Приоритет
ФТ-1	Автоматический импорт CSV-файла при запуске приложения (путь в appsettings.json) и импорт по пути, указанному пользователем	Обязательно
ФТ-2	Загрузка CSV-файла через браузер	Дополнительно
ФТ-3	Проверка заголовков и количества полей в строке	Обязательно
ФТ-4	Валидация полей sku, name, price, stock, is_active, brand, warranty_months	Обязательно
ФТ-5	Пропуск дубликатов артикула с сообщением об ошибке	Обязательно
ФТ-6	Вывод результата: всего строк, импортировано, ошибки, список ошибок по строкам	Обязательно
ФТ-7	Просмотр списка товаров из памяти	Обязательно
ФТ-8	Поиск по названию, фильтр по категории, сортировка по цене и остатку	Дополнительно
ФТ-9	Страница деталей товара, подсчет общего количества и стоимости	Дополнительно
ФТ-10	Экспорт списка товаров в CSV, запись ошибок импорта в журнал	Дополнительно
ФТ-11	Поддержка кодировок UTF-8 и Windows-1251, разделителей «,», «;» и табуляции	Дополнительно

3. АРХИТЕКТУРА РЕШЕНИЯ

3.1. Структура проекта

Решение состоит из проекта веб-приложения и проекта модульных тестов (листинг 1).

Листинг 1 – Структура решения

```
ECommerceCsvImport.sln
├── src/ECommerceCsvImport/          — веб-приложение ASP.NET Core MVC
│   ├── Controllers/ProductsController.cs
│   ├── Models/                      — Product, ImportResult, ImportError,
│   │                                  ImportConstants, ImportMessages,
модели страниц
│   ├── Services/                    — IProductRepository,
InMemoryProductRepository,
│   │                                  IProductImportService,
ProductImportService,
│   │                                  CsvLineParser, ProductCatalogQuery,
│   │                                  ProductCsvExporter,
FileImportErrorLogger
│   ├── Views/Products/              — Import, Index, Details
│   ├── wwwroot/csv/                 — sample_products.csv и папка test
│   ├── appsettings.json             — путь к файлу для импорта при запуске
│   └── Program.cs                   — регистрация сервисов, импорт при
запуске
├── tests/ECommerceCsvImport.Tests/  — модульные тесты xUnit (31 тест)
├── docs/                            — отчет, сопроводительная записка, ТЗ
└── README.md
```

3.2. Роли компонентов

- модель Product описывает товар: артикул, название, описание, цена, валюта, категория, остаток, изображение, активность и поля варианта «Электроника» — бренд, гарантия, номер модели;

- репозиторий InMemoryProductRepository хранит товары в List<Product>, поддерживает добавление, получение списка, поиск по артикулу и очистку; зарегистрирован как Singleton, поэтому данные сохраняются между запросами;

- сервис ProductImportService выполняет всю логику импорта: проверку файла, определение кодировки и разделителя, разбор строк, валидацию, контроль дубликатов, формирование ImportResult;

- контроллер ProductsController принимает запросы, вызывает сервис и передает модель в представление; сам файл не читает и данные не проверяет;

- представления Razor: Import — форма импорта и результат, Index — список товаров, Details — карточка товара.

Схема взаимодействия компонентов приведена на рисунке 1, диаграмма классов — на рисунке 2.

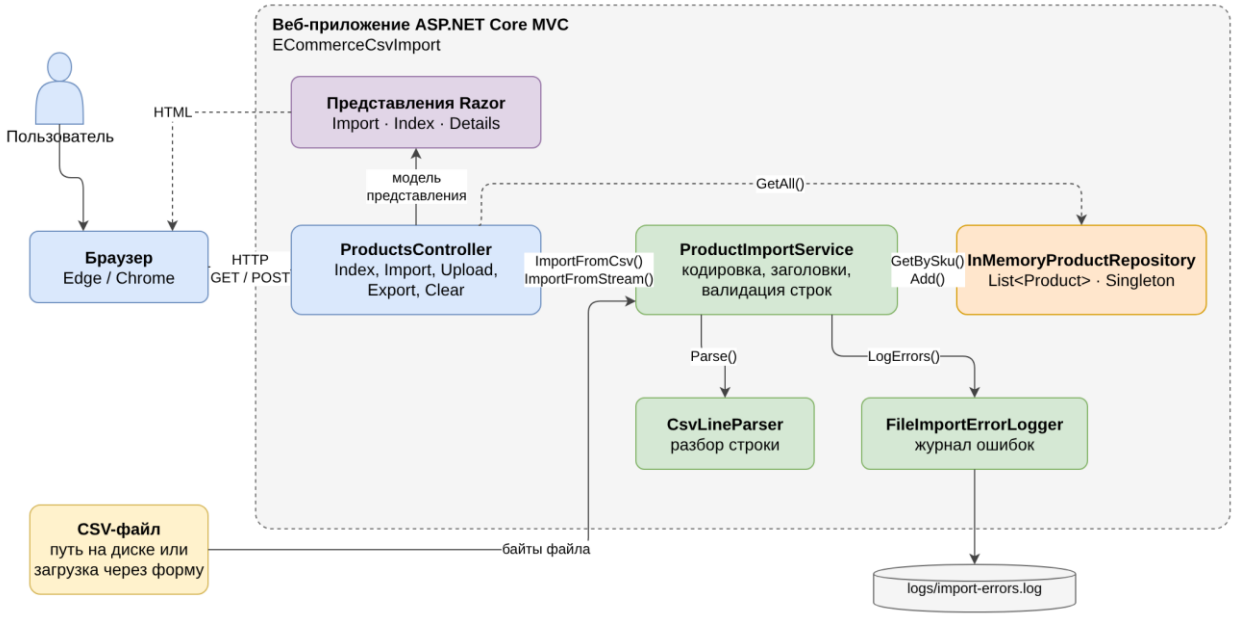


Рисунок 1 – Архитектура приложения

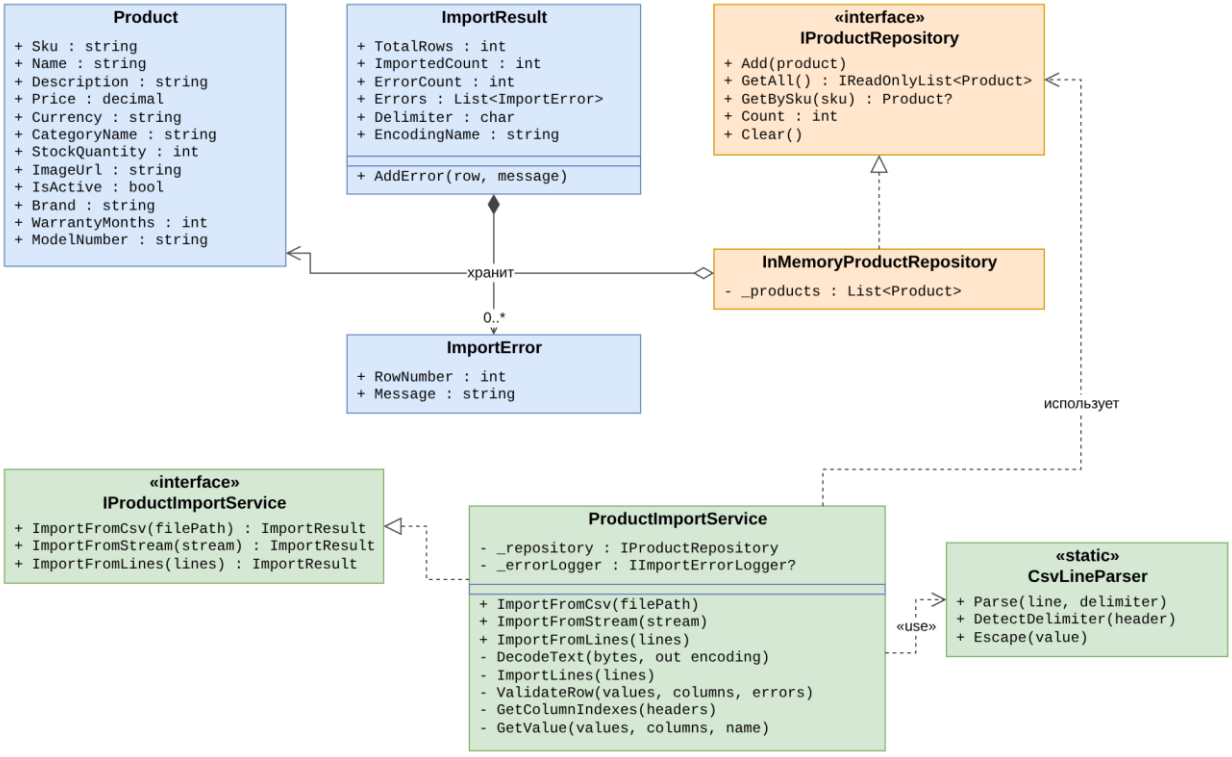


Рисунок 2 – Диаграмма классов модуля

3.3. План разработки

Модуль разрабатывался по итерационной модели: небольшими шагами, после каждого выполнялась проверка и коммит в Git (таблица 3).

Таблица 3 – План разработки модуля импорта

Этап	Содержание работ	Результат
1. Подготовка и анализ	Создание решения, разбор задания, формат CSV, правила проверки	Требования и ТЗ, тестовые CSV-файлы
2. Модель данных	Классы товара и результата импорта, хранилище в памяти	Product, ImportResult, репозиторий
3. Чтение и проверка CSV	Разбор строк с кавычками, валидация полей, контроль дубликатов	Сервис импорта
4. Интеграция в MVC	Контроллер, страницы импорта, списка и карточки товара	Работающее веб-приложение
5. Проверка	Тестовые сценарии, модульные тесты xUnit, отладка	31 пройденный тест
6. Инспектирование	Сверка кода со стандартом C#, вынос констант, оформление README	Код в репозитории Git

Инструменты: Visual Studio 2022, .NET 8, xUnit, Git и GitHub, редактор схем draw.io, браузеры Microsoft Edge и Google Chrome.

4. ФОРМАТ CSV-ФАЙЛА

Пример файла (фрагмент sample_products.csv):

```
sku,name,description,price,currency,category_name,stock,image_url,is_active,b
rand,warranty_months,model_number
1001,Wi-Fi роутер Дом.ру Wave Plus,Роутер Wi-Fi 6 AX3000 для
квартиры,3990,RUB,Роутеры,25,/images/wave-plus.jpg,true,Дом.ру,12,Wave Plus
AX3000
1002,ТВ-приставка Movix Go,Смарт-приставка Android TV 4K,4490,RUB,ТВ-
приставки,40,/images/movix-go.jpg,true,Movix,12,Movix Go
```

Поля файла приведены в таблице 4.

Таблица 4 – Поля CSV-файла

Поле	**Тип данных**	**Обязательность**	**Описание и правило проверки**
sku	string	обязательно	артикул; не пустой, уникальный
name	string	обязательно	название; не пустое
description	string	нет	описание; может быть пустым
price	decimal	обязательно	цена; число больше 0
currency	string	нет	валюта; по умолчанию RUB
category_name	string	нет	категория; по умолчанию «Без категории»
stock	int	обязательно	остаток на складе; целое, не меньше 0
image_url	string	нет	ссылка на изображение
is_active	bool	обязательно	активность; true, false, 1 или 0
brand	string	обязательно	бренд (вариант «Электроника»)
warranty_months	int	обязательно	гарантия; целое от 0 до 60
model_number	string	нет	номер модели; может быть пустым

5. РЕАЛИЗАЦИЯ ФУНКЦИОНАЛЬНОСТИ

5.1. Чтение CSV-файла

Метод ImportFromCsv проверяет, что файл существует, и читает его в массив байтов. Затем определяется кодировка: если файл начинается с метки BOM или корректно читается как UTF-8, используется UTF-8, иначе — Windows-1251. Текст делится на строки. Первая непустая строка считается заголовком: по ней определяется разделитель (запятая, точка с запятой или табуляция) и составляется словарь «название колонки → номер колонки», поэтому порядок колонок в файле может быть любым. Если обязательной колонки нет, выводится ошибка «Неверные заголовки CSV».

Каждая строка разбирается на поля классом CsvLineParser: строка просматривается посимвольно, разделитель внутри кавычек считается частью значения, две кавычки подряд превращаются в одну.

Общий алгоритм импорта приведен на рисунке 3: ошибки в одной строке не останавливают обработку файла — строка пропускается, а ее ошибки добавляются в результат.

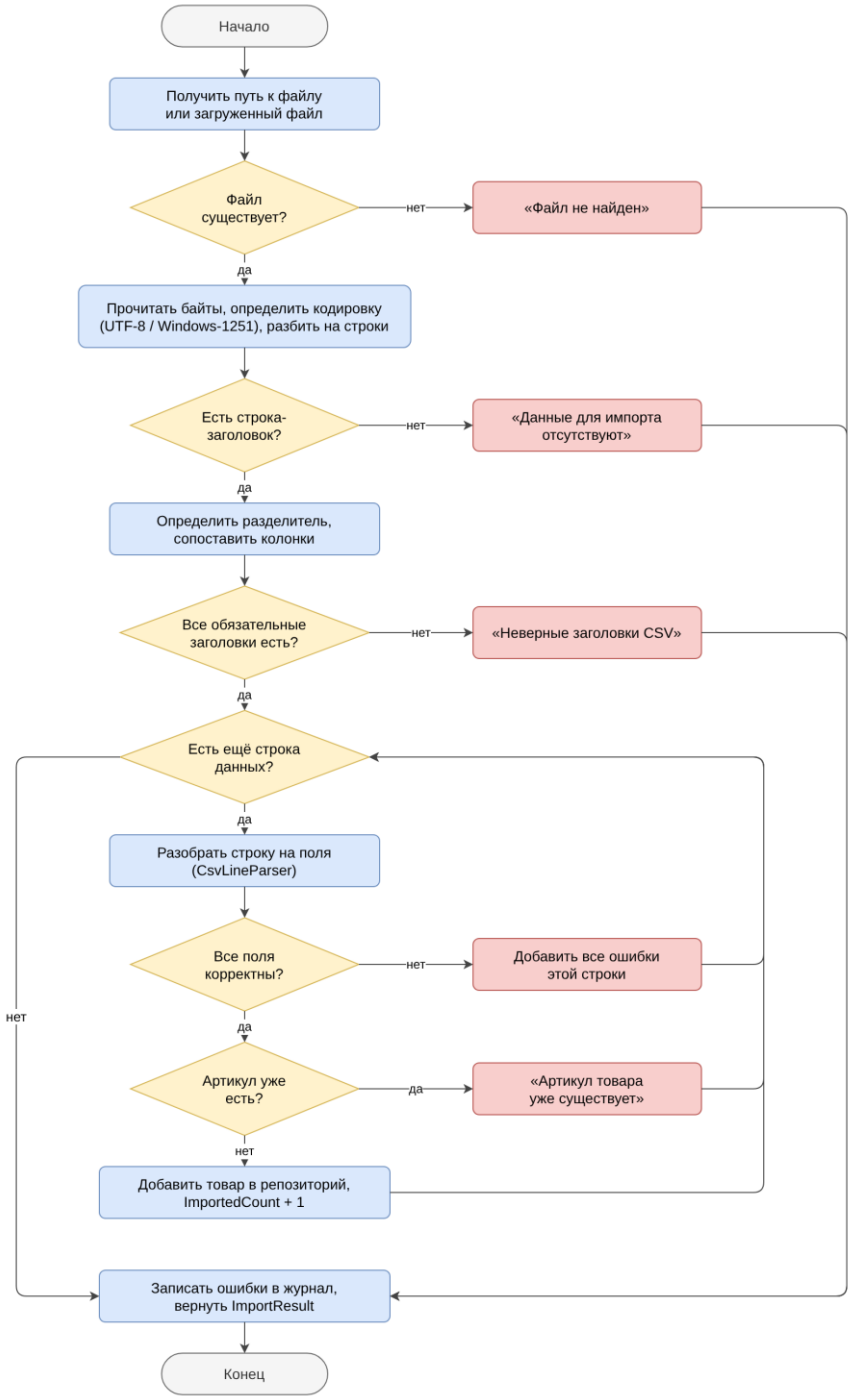


Рисунок 3 – Алгоритм импорта товаров из CSV-файла

5.2. Проверка данных

Все поля строки проверяет метод ValidateRow. Ошибки строки собираются в список, поэтому пользователь сразу видит все проблемы, а не только первую. Фрагмент проверки цены и остатка приведен в листинге 2.

Листинг 2 – Проверка цены и остатка в методе ValidateRow

```
// Цена: допускаем и точку, и запятую как разделитель дробной части
string priceText = GetValue(values, columns, "price").Replace(',', '.');
decimal price;
if (!decimal.TryParse(priceText, NumberStyles.Number,
CultureInfo.InvariantCulture, out price))
    errors.Add(ImportMessages.PriceNotNumber);
else if (price <= 0)
    errors.Add(ImportMessages.PriceNotPositive);
else
    product.Price = price;

int stock;
if (!int.TryParse(GetValue(values, columns, "stock"), out stock))
    errors.Add(ImportMessages.StockNotInteger);
else if (stock < 0)
    errors.Add(ImportMessages.NegativeStock);
else
    product.StockQuantity = stock;
```

Тексты сообщений вынесены в класс ImportMessages, а граничные значения (гарантия от 0 до 60 месяцев, размер файла 5 МБ) — в класс ImportConstants, чтобы в коде не было «магических чисел».

5.3. Добавление товаров в память

Если в строке нет ошибок, товар проверяется на дубликат: артикул ищется среди строк этого же файла и в репозитории. Дубликат пропускается с ошибкой «Артикул товара уже существует», остальные товары добавляются в репозиторий (листинг 3).

Листинг 3 – Контроль дубликатов и добавление товара

```
// 4. Проверка дубликата: в этом файле или уже в памяти
if (importedSkus.Contains(product.Sku) || _repository.GetBySku(product.Sku)
!= null)
{
    result.AddError(rowNumber, ImportMessages.DuplicateSku);
    continue;
}

_repository.Add(product);
importedSkus.Add(product.Sku);
result.ImportedCount++;
```

5.4. Передача результата в представление

Контроллер вызывает сервис, помещает полученный ImportResult в модель страницы и возвращает представление Import (листинг 4). Любое непредвиденное исключение перехватывается, и сообщение выводится на странице — приложение не падает.

Листинг 4 – Действие ImportPost контроллера

```
public IActionResult ImportPost(string? filePath)
{
    ImportViewModel model = CreateImportModel();
    model.FilePath = filePath ?? "";

    if (string.IsNullOrWhiteSpace(filePath))
    {
        model.Result = CreateErrorResult(ImportMessages.FilePathRequired);
        return View("Import", model);
    }

    model.SourceName = filePath;

    try
    {
        model.Result = _importService.ImportFromCsv(GetFullPath(filePath));
    }
    catch (Exception ex)
    {
        // Любая непредвиденная ошибка показывается на странице, приложение
не падает
        model.Result = CreateErrorResult(ImportMessages.UnexpectedError + ":
" + ex.Message);
    }

    return View("Import", model);
}
```

Представление выводит таблицу «Показатель — Значение» (всего строк, успешно импортировано, ошибки) и таблицу ошибок «Строка — Описание ошибки» (рисунок 4). Список товаров передается в представление Index через модель ProductsIndexViewModel (рисунок 5).

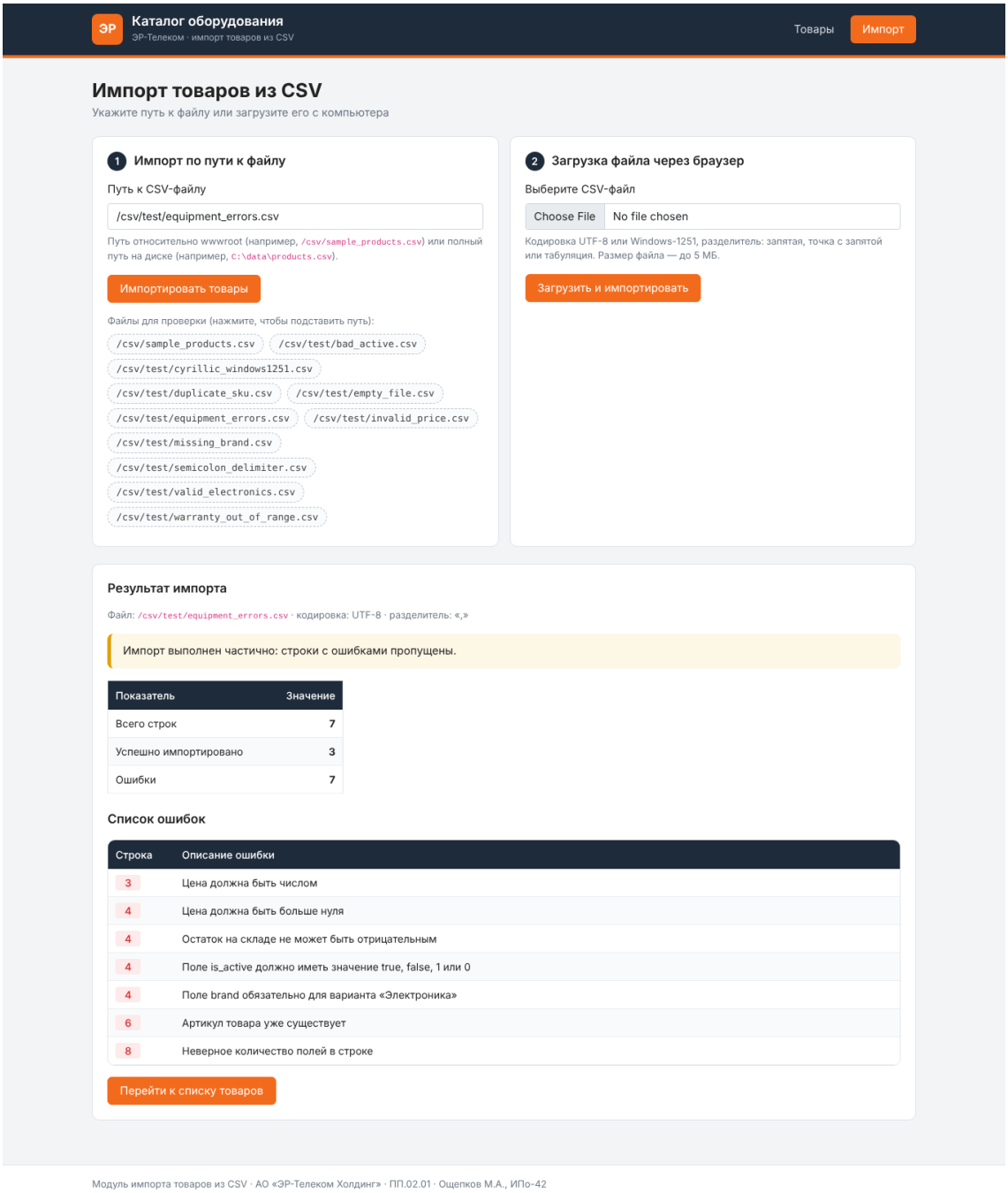


Рисунок 4 – Результат импорта файла с ошибками

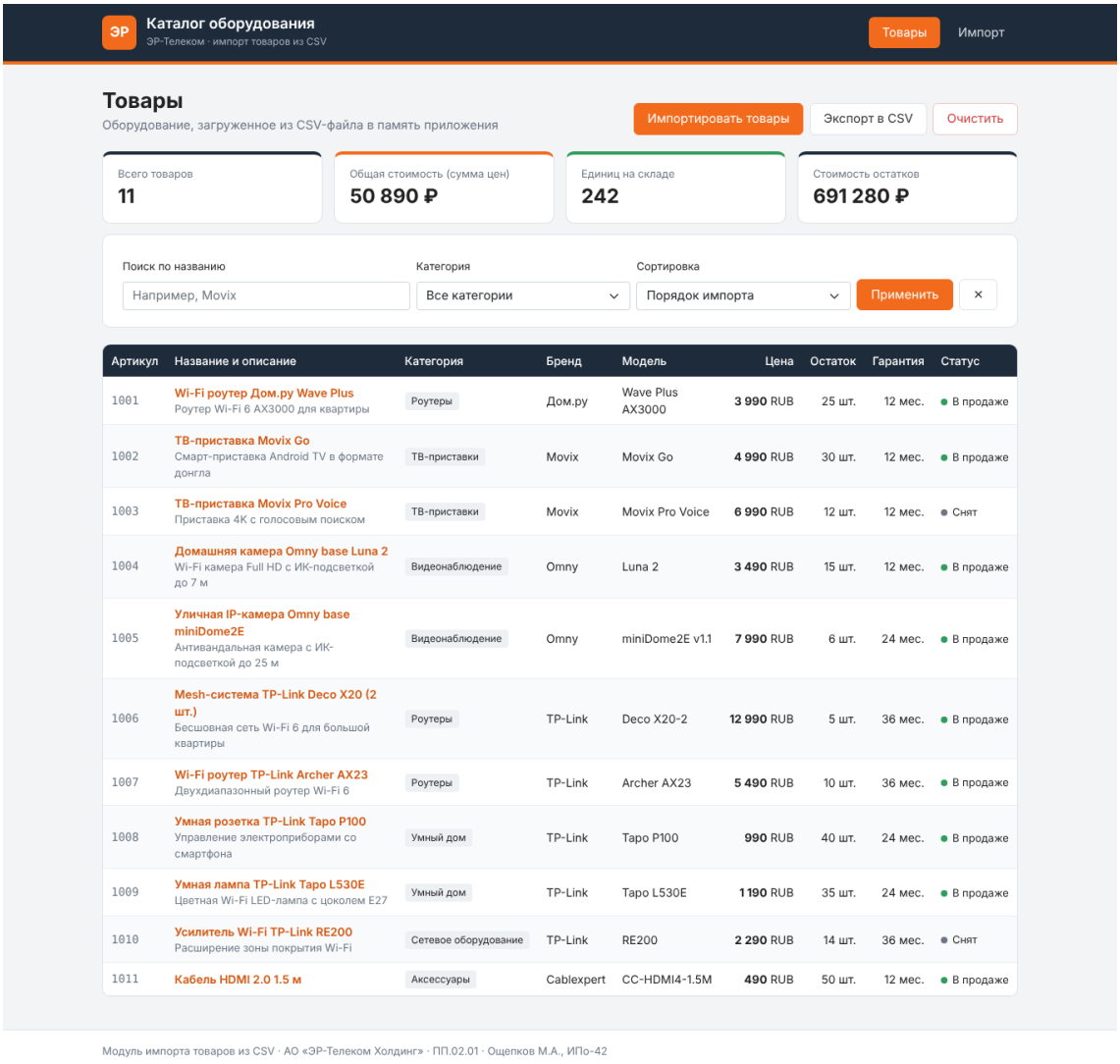


Рисунок 5 – Список товаров

5.5. Импорт при запуске приложения

Сразу после сборки приложения в Program.cs выполняется импорт файла, путь к которому указан в appsettings.json (листинг 5), поэтому после запуска каталог уже заполнен.

Листинг 5 – Импорт при запуске приложения

```
// Автоматический импорт товаров при запуске приложения.
// Путь к файлу (относительно папки wwwroot) задается в appsettings.json,
раздел "Import".
bool autoImport =
builder.Configuration.GetValue<bool>("Import:AutoImportOnStartup");
string? importPath = builder.Configuration["Import:FilePath"];
if (autoImport && !string.IsNullOrEmpty(importPath))
{
    string fullPath = Path.Combine(app.Environment.WebRootPath, importPath);
    IProductImportService importService =
app.Services.GetRequiredService<IProductImportService>();
    ImportResult startupResult = importService.ImportFromCsv(fullPath);
    app.Logger.LogInformation("Импорт при запуске ({Path}): добавлено
{Imported}, ошибок {Errors}",
        importPath, startupResult.ImportedCount, startupResult.ErrorCount);
}
```

6. ТЕСТИРОВАНИЕ И ОТЛАДКА

6.1. Тестовые сценарии

Функциональное тестирование выполнено на тестовых файлах из папки wwwroot/csv/test, включая все сценарии из раздела 8 задания. Перед каждым сценарием список товаров очищался. Результаты приведены в таблице 5.

Таблица 5 – Тестовые сценарии

№	Сценарий	Входные данные	Ожидаемый результат	Фактический результат
1	Успешный импорт	valid_electronics.csv	2 товара добавлены, ошибок нет	Выполнено: 2 / 0
2	Неверная цена	invalid_price.csv	Товар не добавлен, ошибка по строке 2	Выполнено: «Цена должна быть больше нуля»
3	Пустой файл	empty_file.csv	Приложение не падает, 0 товаров	Выполнено: «Данные для импорта отсутствуют»
4	Дубликат артикула	duplicate_sku.csv	Первый товар добавлен, второй пропущен	Выполнено: 1 / 1, «Артикул товара уже существует»
5	Отрицательный остаток	equipment_errors.csv, строка 4	Товар не добавлен	Выполнено: «Остаток на складе не может быть отрицательным»
6	Кириллица Windows-1251	cyrillic_windows1251.csv	Названия отображаются без искажений	Выполнено: 2 / 0
7	Неверное is_active	bad_active.csv	Ошибка по строке 2	Выполнено
8	Нет бренда	missing_brand.csv	Ошибка по строке 2	Выполнено
9	Гарантия вне диапазона	warranty_out_of_range.csv	Ошибка по строке 2	Выполнено
10	Разделитель «;»	semicolon_delimiter.csv	2 товара добавлены	Выполнено: 2 / 0
11	Смешанные ошибки	equipment_errors.csv	Корректные строки добавлены, ошибки по строкам	Выполнено: 3 / 7
12	Файл не найден	/csv/nope.csv	Сообщение об ошибке	Выполнено: «Файл не найден»
13	Повторный импорт	sample_products.csv дважды	Дубликаты пропущены, приложение работает	Выполнено: 0 / 11
14	Файл не CSV	a.txt через браузер	Файл не принят	Выполнено

6.2. Модульные тесты

Модульные тесты написаны на xUnit (таблица 6). Все 31 тест выполняются успешно (рисунок 6).

Таблица 6 – Модульные тесты

Класс тестов	Кол-во	Что проверяется
RepositoryTests	4	добавление, поиск по SKU, поиск без учета регистра, пустой репозиторий
CsvLineParserTests	4	простая строка, пустые поля, запятая в кавычках, экранированные кавычки
ProductImportServiceTests	23	успешный импорт, ошибки файла и заголовков, все правила валидации, дубликаты, значения по умолчанию, кириллица
Итого	31	

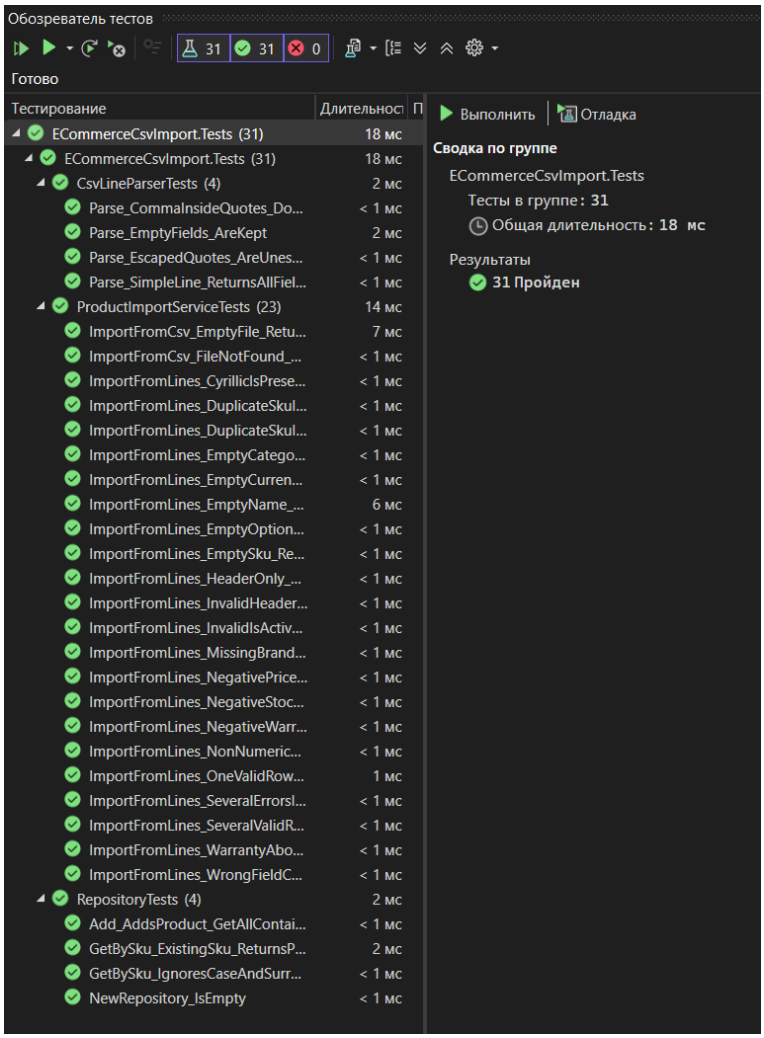


Рисунок 6 – Результат запуска тестов в Visual Studio

6.3. Отладка

Отладка выполнялась средствами Visual Studio: точки останова в методах ImportFromCsv и ValidateRow, пошаговое выполнение, окна «Видимые»,

«Локальные» и «Стек вызовов» (рисунок 7). В стеке видна вся цепочка вызовов: ProductsController.ImportPost → ImportFromCsv → ImportFromBytes → ImportLines → ValidateRow.

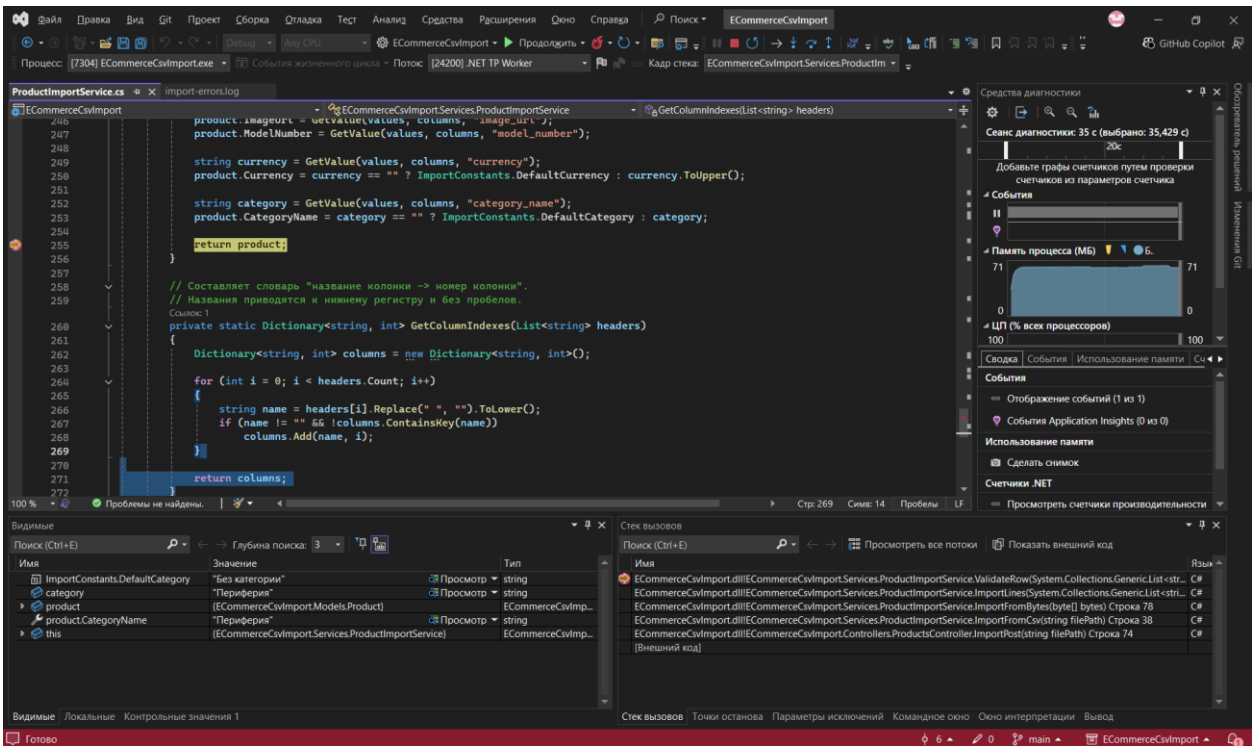


Рисунок 7 – Отладка метода ValidateRow в Visual Studio

В ходе отладки найдены и исправлены проблемы: файл в кодировке Windows-1251 отображался «кракозябрами» — добавлено определение кодировки; строки без последней пустой колонки model_number считались ошибочными — минимальное количество полей теперь считается до колонки warranty_months; товары пропадали между запросами — репозиторий зарегистрирован как Singleton.

6.4. Самопроверка кода

Код проверен по критериям раздела 9 задания (таблица 7).

Таблица 7 – Самопроверка кода

Критерий	Результат проверки
Имена классов	Понятные: ProductImportService, InMemoryProductRepository, CsvLineParser
Имена методов	Отражают действие: ImportFromCsv, ValidateRow, GetColumnIndexes
Обработка ошибок	Неверный CSV не приводит к падению, ошибки выводятся на странице
Разделение ответственности	Контроллер не читает файл и не проверяет данные — это делает сервис
Читаемость и коммента-	Код отформатирован, ключевые методы прокомментирова-

Критерий	Результат проверки
рии	ны
Исключения	Перехватываются в контроллере и при чтении файла
Магические числа	Вынесены в ImportConstants и appsettings.json
Git-коммиты	Осмысленные сообщения, по одному коммиту на этап

История коммитов приведена в листинге 6.

Листинг 6 – История коммитов

```
55e880a Add project scaffold and CSV fixtures
c2b45f3 Add product model and in-memory repository
d937b26 Add CSV line parser with quotes support
e997006 Add product import service with validation for electronics
a9cc995 Add products controller and import views
0d3f077 Add unit tests for repository, parser and import service
64c381f Import sample products on application startup
3ff7ddb Add practice report, cover note and technical specification
a18e6dc Use error messages from test scenarios for price and stock
de7f3fb Update cover note with test scenarios, debugging and conclusions
a09c867 Redesign site pages with custom styles
c08d48c Update screenshots in documentation
9490967 Format technical specification according to GOST 19.201-78 and
19.104-78
8986807 Update final versions of documents
5f22dc6 Separate practice report and cover note
a1c091b Start each cover note section on a new page
```

Репозиторий проекта:

https://github.com/MikhailOshchepkov/ECommerceCsvImport

7. ПОРЯДОК ЗАПУСКА

1. Установить .NET 8 SDK и Visual Studio 2022 (рабочая нагрузка «ASP.NET и разработка веб-приложений»).

2. Скачать репозиторий, открыть ECommerceCsvImport.sln и выбрать проект ECommerceCsvImport как запускаемый.

3. Запустить приложение (Ctrl+F5 или F5). На странице «Товары» сразу видны 11 товаров — файл sample_products.csv импортирован при запуске.

4. Для проверки импорта вручную нажать «Очистить», перейти на страницу «Импорт» и выбрать путь /csv/sample_products.csv или любой файл из /csv/test/.

5. Тесты запускаются через «Тест → Обозреватель тестов → Запустить все тесты» или командой dotnet test.

Примечание. В некоторых версиях Visual Studio при запуске с отладкой (F5) в браузере на движке Chromium отладка может завершаться в момент выбора файла для загрузки. Это ошибка связки отладчика с браузером, к коду модуля она не относится; в этом случае следует запускать без отладки (Ctrl+F5) или выбрать браузер Microsoft Edge.

8. ВЫВОДЫ

В ходе выполнения индивидуального задания разработан модуль импорта товаров из CSV-файлов для варианта «Электроника». Все обязательные требования и все 10 дополнительных элементов задания выполнены, работа модуля подтверждена 14 тестовыми сценариями и 31 модульным тестом.

Полученные умения: разработка требований и технического задания; проектирование ASP.NET MVC-приложения с разделением на контроллер, сервис и репозиторий; работа с CSV-файлами и кодировками; хранение данных в коллекциях в памяти; внедрение зависимостей; отладка средствами Visual Studio; написание модульных тестов; работа с Git.

Возникшие трудности и их решение:

- русские названия в файле Windows-1251 отображались неправильно — реализовано автоматическое определение кодировки;

- разбор строк, в которых значение содержит запятую, — написан собственный парсер с поддержкой кавычек;

- загруженные товары пропадали при переходе между страницами — репозиторий зарегистрирован как Singleton;

- отладка завершалась при выборе файла в браузере Chrome — установлено, что это ошибка Visual Studio, проблема обходится запуском без отладки или через Edge.

Что можно улучшить в дальнейшем: хранение товаров в базе данных (Entity Framework Core), асинхронный импорт больших файлов, постраничный вывод списка, обновление существующих товаров при повторном импорте, загрузка изображений товаров, предварительный просмотр файла перед импортом.